

This is unreleased documentation for Serverpod **Next** version.

For up-to-date documentation, see the [latest version](#) (4.0.0).

3..20..15

SETUP

 Copy as Markdown

Set up Facebook sign-in

Sign in with Facebook uses a **Facebook Login app** and the external `serverpod_auth_idp_flutter_facebook` package. Unlike providers built into the core auth module, Facebook sign-in needs this extra package, which wraps the native Facebook SDK to handle the flow on iOS, Android, web, and macOS.

Prerequisites

Before following this guide, make sure you have:

- A Facebook account with access to [Facebook for Developers](#).
- A running Serverpod project (server, client, and Flutter app packages from `serverpod create`).
- The Serverpod auth module installed and configured per the [authentication setup](#). If your project was generated with an older Serverpod version, follow that guide first to add `serverpod_auth_idp_server` and `serverpod_auth_idp_flutter` and to configure `pod.initializeAuthServices()` before continuing.

Create your Facebook app

1. Go to [Facebook for Developers](#) and log in with your Facebook account.
2. Click **Create an app** and follow the setup wizard:
 - **App details:** Enter your **App name** and **App contact email**.



Create an app

☒ App details
 ☐ Use cases
 ☐ Business
 ☐ Requirements
 ☐ Overview

App name
This is the app name that will show on your My Apps page and associated with your app ID. You can change the name later in Settings.

App contact email
This is the email address we'll use to contact you about your app. Make sure it is an address you check regularly. We may contact you about policies, app restrictions or recovery if your app is deleted or compromised.

- **Use cases:** Select "**Authenticate and request data from users with Facebook Login**".

☒ **Authenticate and request data from users with Facebook Login**
 Our most common use case. A secure, fast way for users to log into your app or game and for the app to ask for permissions to access their data to personalize their experience. [About Facebook Login](#)

☐ **Connect with customers through WhatsApp**
 Start a WhatsApp conversation, send notifications, create ads that click-to-WhatsApp and provide support. Business portfolio required. [About WhatsApp Business Platform](#)

☐ **Some use cases can't be combined on the same app. Create another app if you need to add use cases that aren't combinable.**

1 use case added

- **Business:** Optionally select a **Business Portfolio** if available.
- Complete any additional **Requirements**.

3. Click through to the **Overview** page and go to your app **Dashboard**.

Configure Facebook Login

After creating your app, click the **Customize** button next to "**Authenticate and request data from users with Facebook Login**" on the **Dashboard** or **Use cases** page. This opens the customization page with the following menu options:

Use cases

Select each use case to customize how it works and add extensions for more functionality.

[Add use cases](#)

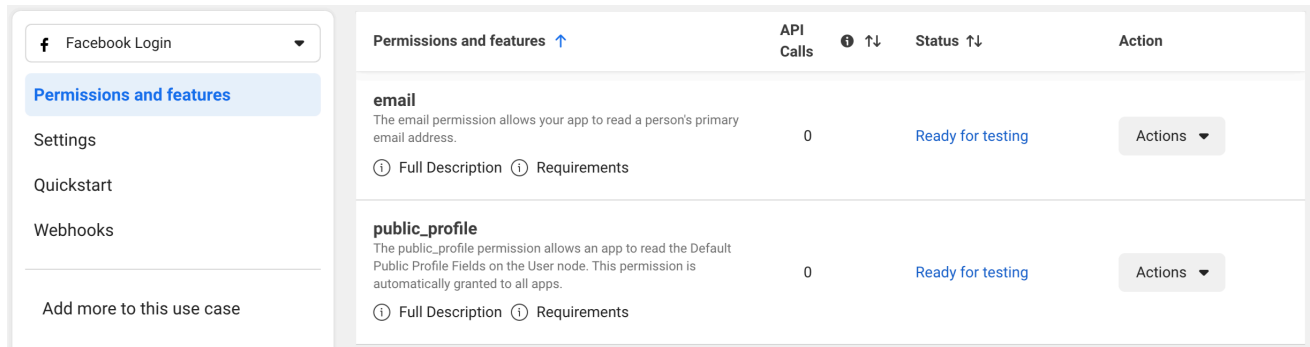
☒ **Authenticate and request data from users with Facebook Login**
 Our most common use case. A secure, fast way for users to log into your app or game and for the app to ask for permissions to access their data to personalize their experience. [About Facebook Login](#)

[Customize](#)

1. Permissions and features

Configure the permissions your application will request:

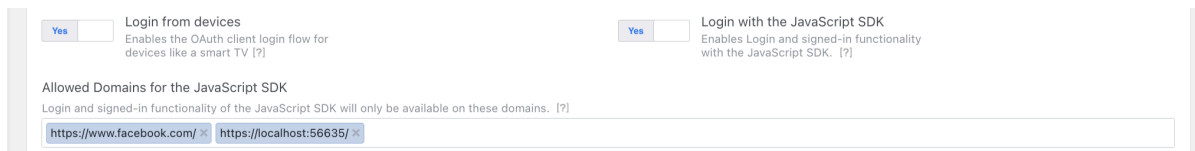
- **public_profile**: Added by default, allows access to basic profile information.
- **email**: Recommended. Allows to read a person's primary email address.



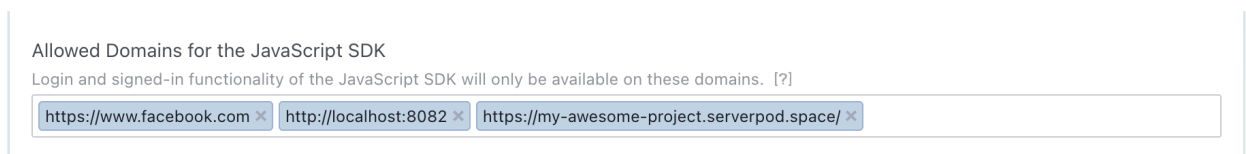
2. Settings

Configure the authentication settings:

- Turn **Login with the JavaScript SDK** to **Yes**. Web and macOS sign-in use the Facebook JavaScript SDK, and the flow fails at runtime with "JSSDK Option is Not Toggled" if this is left off. You can skip this if you only target iOS and Android.



- In **Allowed Domains for the JavaScript SDK**, add:
 - **https://www.facebook.com** (required to avoid CORS issues)
 - Your own domain if you plan to support web authentication (for example, **https://yourdomain.com**)



3. Quickstart

Set up your platforms by selecting and configuring:

- **iOS**
- **Android**
- **Web**
- **Other** (for additional platforms like macOS)

Save your changes after completing the configuration.

Get app credentials

1. Go to **App settings** > **Basic** in your Facebook App Dashboard to retrieve:
 - **App ID**
 - **App secret**
2. Go to **App settings** > **Advanced** to retrieve:
 - **Client token**

TIP

The **App secret** is sensitive. Keep it confidential and never commit it to version control. The **Client token** is used by the mobile SDKs on Android and iOS. Web and macOS initialize with the App ID alone.

Server-side configuration

Store your credentials

Your server's `config/passwords.yaml` already has `development:`, `staging:`, and `production:` sections from the project template. Add `facebookAppId` and `facebookAppSecret` to the `development:` section using the values from your Facebook app:

```
development:
  # ... existing keys (database, redis, serviceSecret, etc.) ...
  facebookAppId: 'your-facebook-app-id'
  facebookAppSecret: 'your-facebook-app-secret'
```

For production, add the same two keys to the `production:` section, or set the `SERVERPOD_PASSWORD_facebookAppId` and `SERVERPOD_PASSWORD_facebookAppSecret` environment variables on your production server. See [Publishing to production](#) below for the full prod walkthrough.

WARNING

Never commit `config/passwords.yaml` **to version control.** It contains your app secret. Use environment variables or a secrets manager in production.

Add the Facebook identity provider

Your server's `server.dart` file (for example, `my_project_server/lib/server.dart`) should already contain a `pod.initializeAuthServices()` call if your project was created with the Serverpod project template (`serverpod create`). If it's not there, see [Setup](#) first to configure the auth module and JWT settings.

Add the Facebook import and `FacebookIdpConfigFromPasswords()` to the existing `identityProviderBuilders` list:

```
import 'package:serverpod_auth_idp_server/providers/facebook.dart';
```

```
pod.initializeAuthServices(  
  tokenManagerBuilders: [  
    JwtConfigFromPasswords(),  
  ],  
  identityProviderBuilders: [  
    // ... any existing providers (e.g., EmailIdpConfigFromPasswords) ...  
    FacebookIdpConfigFromPasswords(),  
  ],  
);
```

The `FacebookIdpConfigFromPasswords()` constructor automatically loads the App ID and secret from the `facebookAppId` and `facebookAppSecret` keys in `config/passwords.yaml` (or the matching `SERVERPOD_PASSWORD_*` environment variables).

TIP

If you need more control over how the credentials are loaded, use `FacebookIdpConfig(appId: ..., appSecret: ...)` instead. See [Customizations](#) for details.

Create the endpoint

Create a new endpoint file in your server project (for example, `my_project_server/lib/src/auth/facebook_idp_endpoint.dart`) alongside the existing auth endpoints. Extending the base class registers the sign-in methods with your server so your app can call them to complete the authentication flow:

```
import 'package:serverpod_auth_idp_server/providers/facebook.dart';
```

```
class FacebookIdpEndpoint extends FacebookIdpBaseEndpoint {}
```

Start the server

Start the server from your server project directory (for example, `my_project_server/`):

```
serverpod start
```

Then create and apply the migration for the provider's tables: in the `serverpod start` terminal, press **M** to create and apply the migration.

WARNING

Skipping the migration will cause the server to crash at runtime when the Facebook provider tries to read or write user data. More detailed instructions can be found in the general [identity providers setup section](#).

Client-side configuration

Install required packages

Add the `serverpod_auth_idp_flutter_facebook` package to your Flutter project:

```
flutter pub add serverpod_auth_idp_flutter_facebook
```

This package uses `flutter_facebook_auth` to handle the authentication flow. The documentation for that package should in most cases also apply to this setup.

INFO

Unlike providers such as Google or Apple that are built into the core `serverpod_auth_idp_flutter` package, Facebook authentication is provided as a separate external package. This keeps the core package lightweight while allowing you to opt-in to Facebook authentication when needed.

Android

The minimum Android SDK version required is 21. Update your `android/app/build.gradle` file:

```
defaultConfig {  
    ...  
    minSdkVersion 21
```

```
...  
}
```

Add Facebook SDK configuration

1. Open or create `/android/app/src/main/res/values/strings.xml`:

```
<?xml version="1.0" encoding="utf-8"?>  
<resources>  
  <string name="facebook_app_id">YOUR_FACEBOOK_APP_ID</string>  
  <string name="fb_login_protocol_scheme">fbYOUR_FACEBOOK_APP_ID</string>  
  <string name="facebook_client_token">YOUR_CLIENT_TOKEN</string>  
</resources>
```

Replace `YOUR_FACEBOOK_APP_ID` with your App ID and `YOUR_CLIENT_TOKEN` with your Client token (found in **Settings > Advanced > Client token** in the Facebook App Dashboard).

2. Open `/android/app/src/main/AndroidManifest.xml` and add the following:

Add internet permission before the `<application>` element:

```
<uses-permission android:name="android.permission.INTERNET"/>
```

Add meta-data inside the `<application>` element:

```
<application>  
  ...  
  <meta-data  
    android:name="com.facebook.sdk.ApplicationId"  
    android:value="@string/facebook_app_id"/>  
  <meta-data  
    android:name="com.facebook.sdk.ClientToken"  
    android:value="@string/facebook_client_token"/>  
  ...  
</application>
```

Configure Package Visibility (Android 11+)

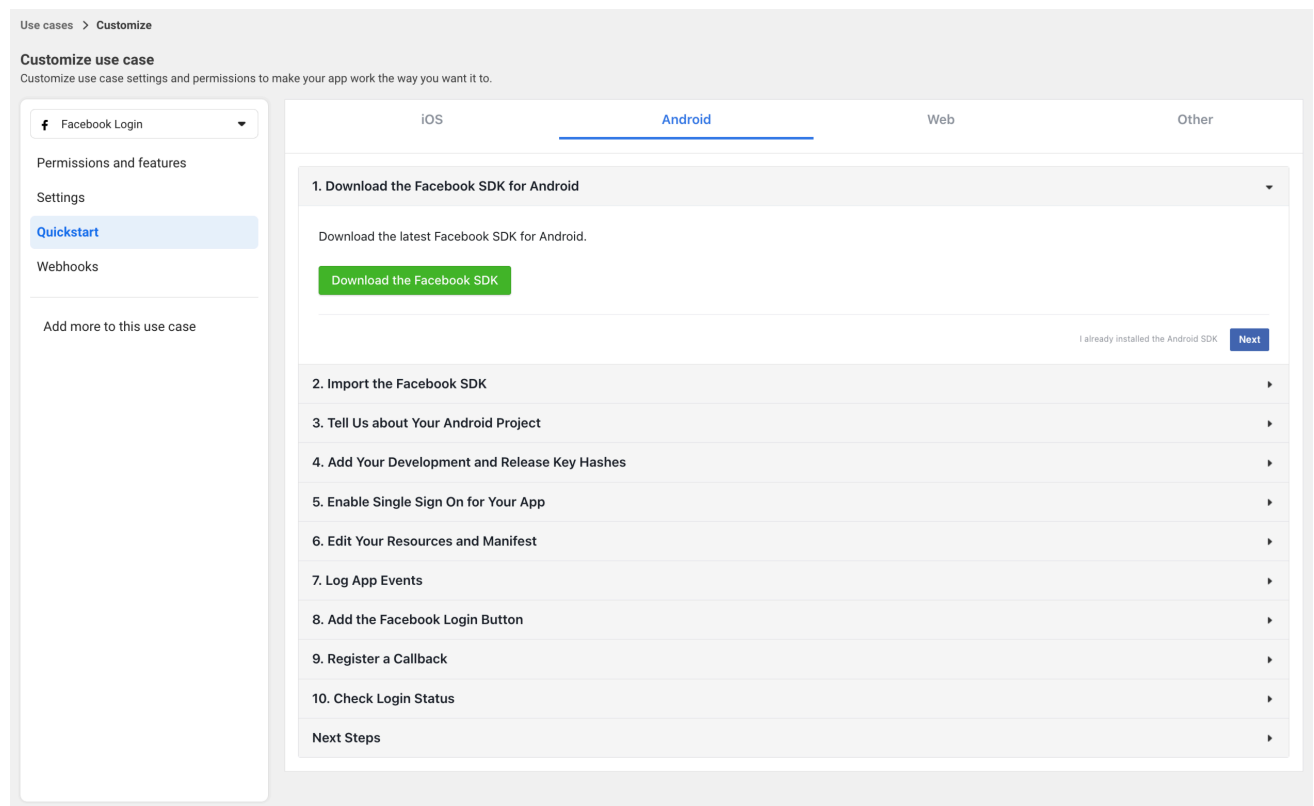
For Android 11 and above, add the following inside the `<manifest>` element:

```
<manifest package="com.example.app">  
  <queries>  
    <provider android:authorities="com.facebook.katana.provider.PlatformProvider" />  
  </queries>  
  ...  
</manifest>
```

</manifest>

Configure Android platform in Facebook

In the Facebook App Dashboard, go to **Use cases** > **Customize** > **Quickstart** and select the **Android** tab.



Follow these steps:

Step 3: Tell Us about Your Android Project

Fill in your Android app details:

- **Package Name:** Your app's package identifier (found in `android/app/build.gradle` as `applicationId` or in `AndroidManifest.xml` as `package`).
- **Class Name:** Your main activity class name (typically `MainActivity`).

Package Name

Your package name uniquely identifies your Android app. We use this to let people download your app from Google Play if they don't have it installed. You can find this in your Android Manifest or your app's build.gradle file.

Default Activity Class Name

This is the fully qualified class name of the activity that handles deep linking such as com.example.app.DeepLinkingActivity. We use this when we deep link into your app from the Facebook app. You can also find this in your Android Manifest.

Save

Step 4: Add Your Development and Release Key Hashes

Generate and add your key hashes:

Debug Key Hash (for development):

macOS/Linux:

```
keytool -exportcert -alias androiddebugkey -keystore ~/.android/debug.keystore | openssl  
sha1 -binary | openssl base64
```

Windows:

```
keytool -exportcert -alias androiddebugkey -keystore  
"C:\Users\USERNAME\.android\debug.keystore" | "PATH_TO_OPENSSL_LIBRARY\bin\openssl" sha1  
-binary | "PATH_TO_OPENSSL_LIBRARY\bin\openssl" base64
```

Release Key Hash (for production):

```
keytool -exportcert -alias YOUR_RELEASE_KEY_ALIAS -keystore YOUR_RELEASE_KEY_PATH |  
openssl sha1 -binary | openssl base64
```

Paste the generated key hashes into the Facebook console and save.

Key Hashes

epW3BT5v7m56YMhx2vXdrxznFrg= ✕

Save

NOTE

You can skip the remaining steps (1, 2, 5, 6+) as they are not required for Flutter apps or have already been covered.

For more detailed Android setup instructions, refer to the [flutter_facebook_auth Android documentation](#).

iOS

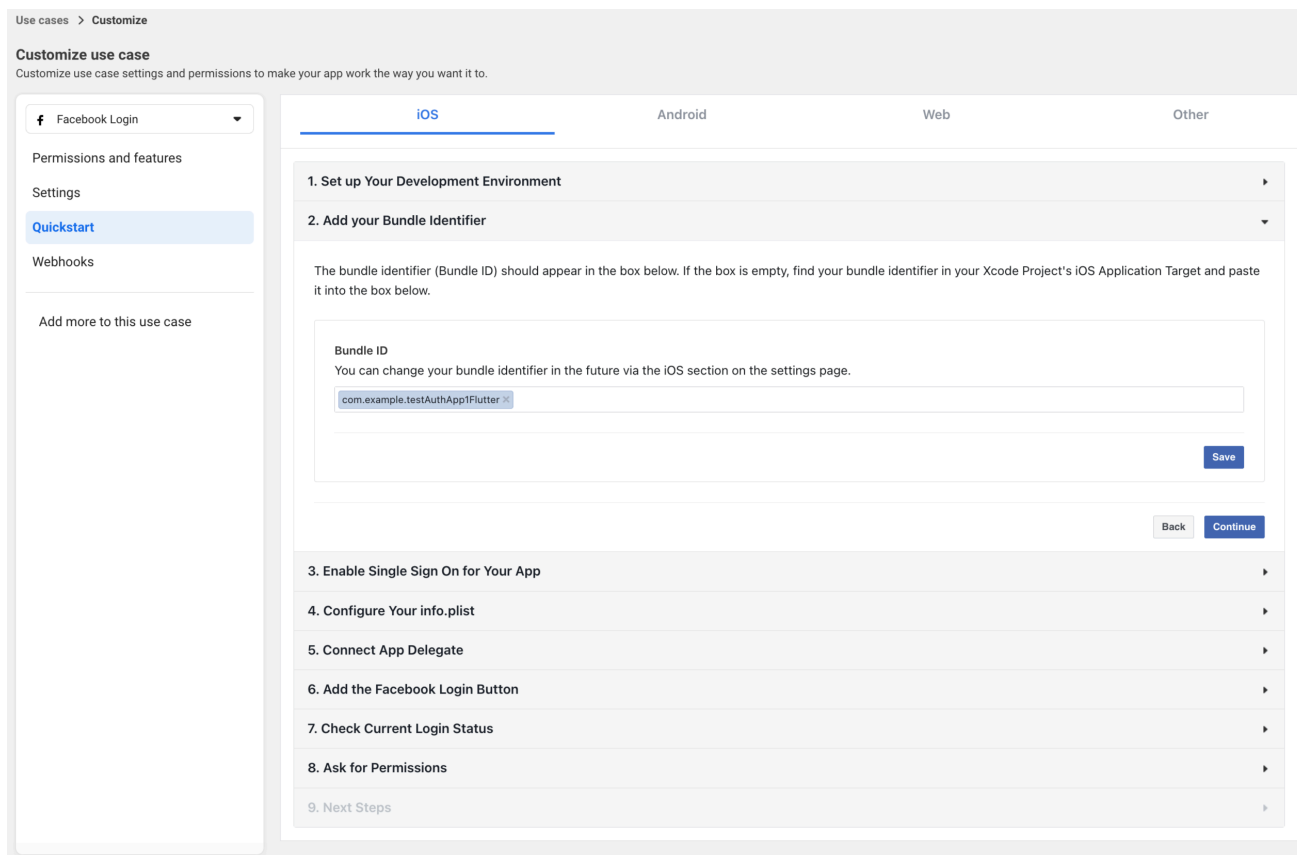
The minimum iOS deployment target is 12.0. Update your `ios/Podfile`:

```
platform :ios, '12.0'
```

Also set the deployment target to 12.0 in Xcode (**Runner** > **General** > **Deployment Info**).

Configure iOS platform in Facebook

In the Facebook App Dashboard, go to **Use cases** > **Customize** > **Quickstart** and select the **iOS** tab.



Follow these steps:

Step 2: Add your Bundle Identifier

Enter your iOS app's Bundle Identifier (found in Xcode under **Runner** > **General** > **Identity** or in your **Info.plist**).

Bundle ID
You can change your bundle identifier in the future via the iOS section on the settings page.

com.example.testAuthApp1Flutter ✕

Save

Step 4: Configure Your info.plist

Open **ios/Runner/Info.plist** and add the following configuration (replace placeholders with your actual values):

```
<key>CFBundleURLTypes</key>
<array>
  <dict>
```

```

    <key>CFBundleURLSchemes</key>
    <array>
      <string>fbYOUR_APP_ID</string>
    </array>
  </dict>
</array>
<key>FacebookAppID</key>
<string>YOUR_APP_ID</string>
<key>FacebookClientToken</key>
<string>YOUR_CLIENT_TOKEN</string>
<key>FacebookDisplayName</key>
<string>YOUR_APP_NAME</string>
<key>LSApplicationQueriesSchemes</key>
<array>
  <string>fbapi</string>
  <string>fb-messenger-share-api</string>
</array>

```

Replace `YOUR_APP_ID` with your Facebook App ID, `YOUR_CLIENT_TOKEN` with your Client token, and `YOUR_APP_NAME` with your app's name.

TIP

If you have other providers (like Google) implemented, merge the values for `CFBundleURLTypes` and `LSApplicationQueriesSchemes` instead of creating duplicate keys.

Additionally, ensure that the **Keychain Sharing** capability is enabled for your target in Xcode (**Runner** > **Signing & Capabilities** > + **Capability** > **Keychain Sharing**).

NOTE

You can skip the remaining steps (1, 3, 5-9) as they are not required for Flutter apps or have already been covered.

iOS App Tracking Transparency (ATT)

On iOS, Facebook may issue **limited access tokens** when App Tracking Transparency (ATT) permission is not granted. These limited tokens cannot be validated by the server or used to retrieve user profile data, which will cause authentication to fail.

To ensure full Facebook authentication functionality on iOS, you should request ATT permissions before initiating Facebook sign-in. You can use the `app_tracking_transparency` package to handle this:

```

import 'package:app_tracking_transparency/app_tracking_transparency.dart';

// Request tracking authorization before showing Facebook sign-in

```

```
final status = await AppTrackingTransparency.requestTrackingAuthorization();
```

⚠️ WARNING

Without ATT permission granted, Facebook authentication fails on iOS. Consider requesting this permission early in your app's flow or before showing the Facebook sign-in button.

For more detailed iOS setup instructions, refer to the [flutter_facebook_auth iOS documentation](#).

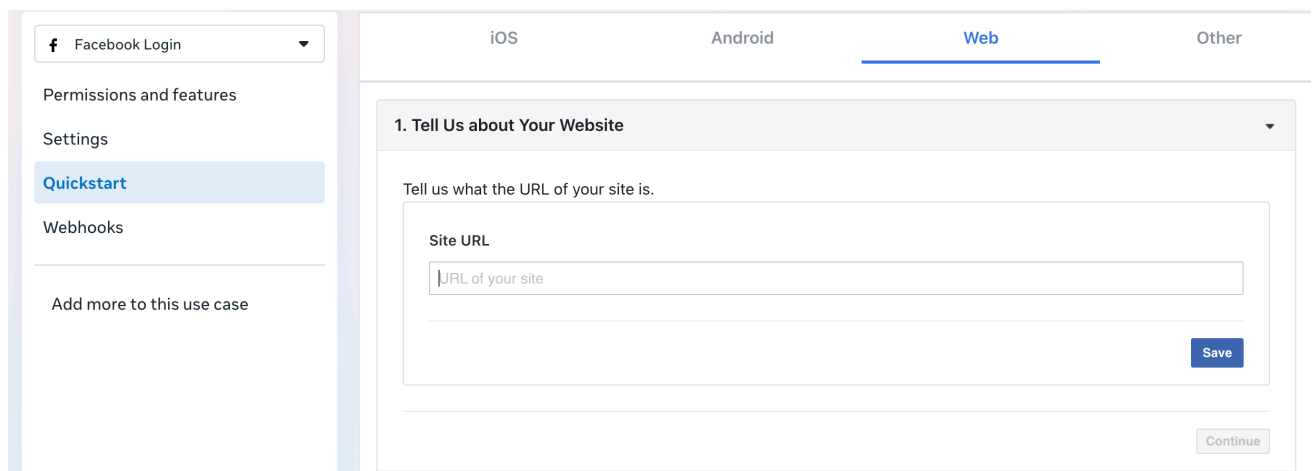
Web

For web platforms, you must initialize the FacebookSignInService with your App ID:

```
await client.auth.initializeFacebookSignIn(  
  appId: 'YOUR_FACEBOOK_APP_ID',  
);
```

Configure Web platform in Facebook

In the Facebook App Dashboard, go to **Use cases** > **Customize** > **Quickstart** and select the **Web** tab.



The screenshot shows the Facebook App Dashboard interface. On the left is a sidebar with a dropdown menu set to 'Facebook Login'. Below it are links for 'Permissions and features', 'Settings', 'Quickstart' (which is highlighted in blue), and 'Webhooks'. At the bottom of the sidebar is a link 'Add more to this use case'. The main content area has tabs for 'iOS', 'Android', 'Web' (which is selected and underlined), and 'Other'. Under the 'Web' tab, there is a section titled '1. Tell Us about Your Website'. Below this title is a prompt 'Tell us what the URL of your site is.' followed by a text input field labeled 'Site URL' with the placeholder text 'URL of your site'. To the right of the input field is a blue 'Save' button. At the bottom right of the section is a grey 'Continue' button.

Follow these steps:

1. Add Site URL in Quickstart

Enter your website's Site URL (for example, <https://yourdomain.com>) and save your changes.

📝 NOTE

You can skip the remaining steps (2-5) as they are not required for Flutter apps or have already been covered.

2. Configure Settings (Important)

Go to **Use cases** > **Customize** > **Settings** and configure the following:

- Confirm **Login with the JavaScript SDK** is set to **Yes** (from the [Settings step](#) above). Web sign-in fails with "JSSDK Option is Not Toggled" without it.
- Under **Allowed Domains for the JavaScript SDK**, add all domains where your app will be accessible:
 - Your development domain: `http://localhost:PORT` (for example, `http://localhost:8082` when serving the Flutter web build from Serverpod).
 - Your production domain: `https://yourdomain.com`.

This enables Facebook authentication on these domains. Without it, Facebook sign-in fails due to CORS restrictions.

Allowed Domains for the JavaScript SDK

Login and signed-in functionality of the JavaScript SDK will only be available on these domains. [?]

`https://www.facebook.com` `http://localhost:8082` `https://my-awesome-project.serverpod.space/`

⚠ WARNING

The Facebook JavaScript SDK is only allowed over `https`, but you can test the plugin on `localhost` (you will see an error message in your web console).

For more detailed web setup instructions, refer to the [flutter_facebook_auth web documentation](#).

macOS

Enable Network Capabilities

In `macos/Runner/Info.plist`, add:

```
<key>com.apple.security.network.server</key>
<true/>
```

In Xcode, select the **Runner** target, go to **Signing & Capabilities**, and enable **Outgoing Connections**.

Enable Keychain Sharing

macOS uses `flutter_secure_storage` to securely store session data. Add the **Keychain Sharing** capability in Xcode (**Runner** > **Signing & Capabilities** > + **Capability** > **Keychain Sharing**). See [Set up authentication on macOS](#) for file-based setup steps and troubleshooting.

Initialize the FacebookSignInService

For macOS, you must initialize the FacebookSignInService with your App ID:

```
await client.auth.initializeFacebookSignIn(  
  appId: 'YOUR_FACEBOOK_APP_ID',  
);
```

Configure macOS platform in Facebook

1. Configure settings

Go to **Use cases > Customize > Settings** and configure the following:

- Enable **Login from Devices** under **Client OAuth settings**.
- Enable **Login with the JavaScript SDK** under **Client OAuth settings**.
- In **Allowed Domains for the JavaScript SDK**, add `https://www.facebook.com` to avoid CORS issues.

The screenshot shows the 'Customize use case' settings for 'Facebook Login'. The left sidebar contains a navigation menu with 'Settings' selected. The main content area is divided into several sections:

- Redirect URI Validator:** A section with a text input for 'Redirect URI to check' containing 'https://example.com/oauth.php' and a 'Check URI' button.
- Client OAuth settings:** A section with multiple toggle switches:
 - Client OAuth login:** Enabled (Yes).
 - Web OAuth login:** Enabled (Yes).
 - Force Web OAuth reauthentication:** Disabled (No).
 - Use Strict Mode for redirect URIs:** Enabled (Yes).
 - Enforce HTTPS:** Enabled (Yes).
 - Embedded browser OAuth login:** Enabled (Yes).
- Valid OAuth Redirect URIs:** A section with a text input for 'Valid OAuth redirect URIs' and a 'Copy to clipboard' button.
- Login from devices:** Enabled (Yes).
- Login with the JavaScript SDK:** Enabled (Yes).
- Allowed Domains for the JavaScript SDK:** A section with a text input containing 'https://www.facebook.com/' and 'https://localhost:56635/'.

For more detailed macOS setup instructions, refer to the [flutter_facebook_auth macOS documentation](#).

Present the authentication UI

Initialize the Facebook sign-in service

Initialize the service in your app's `lib/client.dart` using the `initializeFacebookSignIn()` extension method on `FlutterAuthSessionManager`, on the line after the existing `client.auth.initialize()` call in `initializeClient()`.

```
import
'package:serverpod_auth_idp_flutter_facebook/serverpod_auth_idp_flutter_facebook.dart';

Future<void> initializeClient() async {
  client = Client(await serverUrl)
    ..connectivityMonitor = FlutterConnectivityMonitor()
    ..authSessionManager = FlutterAuthSessionManager();

  unawaited(client.auth.initialize());
  client.auth.initializeFacebookSignIn();
}
```

NOTE

For web and macOS platforms, you must provide your Facebook App ID during initialization:

```
client.auth.initializeFacebookSignIn(
  appId: 'YOUR_FACEBOOK_APP_ID',
);
```

For iOS and Android, the App ID is not required as the SDK reads credentials from native configuration.

Show the Facebook sign-in button

If you use the template's `SignInWidget` (see [Present the authentication UI](#)), the Facebook button is detected and shown automatically once the `serverpod_auth_idp_flutter_facebook` package is installed and the service is initialized. It handles the full sign-in flow, token management, and error handling on iOS, Android, web, and macOS.

To customize the button or build a fully custom UI, see [Customizations](#).

Publishing to production

Before going live, complete the following steps.

1. Switch the app to Live mode

New Facebook apps start in **Development mode**, where only people with a [role on the app](#) (admins, developers, testers) can sign in. To let anyone sign in, toggle the app from **Development** to **Live** at the top of the [Facebook App Dashboard](#).

NOTE

Going Live requires a valid **Privacy Policy URL** (**App settings** > **Basic**), and requesting permissions beyond `email` and `public_profile` requires **App Review**. See [Accessing Facebook APIs](#).

2. Add your production domains and platforms

- **Web and macOS:** In **Use cases** > **Customize** > **Settings**, confirm **Login with the JavaScript SDK** is **Yes** and add your production domain to **Allowed Domains for the JavaScript SDK** (for example, `https://yourdomain.com`) alongside your development domain. Both can stay registered so dev and prod work at the same time.
- **Android:** Add your **release key hash** (not only the debug one) to the Android platform in the Facebook app. Generate it from your release keystore:

```
keytool -exportcert -alias YOUR_RELEASE_KEY_ALIAS -keystore YOUR_RELEASE_KEY_PATH |  
openssl sha1 -binary | openssl base64
```

- **iOS:** No change is needed between dev and prod for the custom URL scheme.

3. Set production credentials

Production runs out of the `production:` section of `passwords.yaml`, which is separate from the `development:` section you populated during setup. Adding production credentials does not replace your development ones. Both stay in place, and Serverpod picks the right set based on the run mode.

You can reuse the same Facebook app for development and production, or [create a separate app](#) per environment and use its credentials.

Self-hosted

Add `facebookAppId` and `facebookAppSecret` to the `production:` section of `passwords.yaml`:

```
production:  
  # ... existing keys ...  
  facebookAppId: 'your-facebook-app-id'  
  facebookAppSecret: 'your-facebook-app-secret'
```

Alternatively, set the `SERVERPOD_PASSWORD_facebookAppId` and `SERVERPOD_PASSWORD_facebookAppSecret` [environment variables](#) on your production server with the same values.

Serverpod Cloud

Use `serverpod cloud password set` to upload each value. The App ID is public, so pass it as a positional argument. The App secret is sensitive, so read it from a file with `--from-file` to keep it out of your shell history:

```
serverpod cloud password set facebookAppId your-facebook-app-id
serverpod cloud password set facebookAppSecret --from-file path/to/facebook-app-secret.txt
```

Run these from your linked server project directory, or pass `--project <project-id>` on each call. See the [Serverpod Cloud passwords guide](#) for project linking and other options.

4. Provide the App ID to the production Flutter build

On web and macOS the App ID must be available to the Flutter app at build time. When you omit `appId`, the provider reads it from the `FACEBOOK_APP_ID` compile-time environment variable, so a single `client.dart` works in dev and prod:

```
await client.auth.initializeFacebookSignIn();
```

Pass the value in your production build (or the `flutter_build` step for Serverpod Cloud):

```
flutter build web --dart-define=FACEBOOK_APP_ID=your-facebook-app-id
```

On iOS and Android the App ID comes from the native configuration files, so no build-time value is needed. See [Configuring Facebook sign-in on the app](#).

TIP

If you run into issues, see the [troubleshooting guide](#).